Bien sûr Loïc — je vais te montrer **le déroulement complet**, étape par étape, depuis **l'action dans ton interface Vue.js**, jusqu'à **l'insertion finale dans ta base MySQL via ton API REST PHP**.

Je vais te raconter ça comme si on suivait la requête en temps réel, du clic utilisateur jusqu'au `INSERT INTO`.

🔥 Vue d'ensemble du parcours d'une requête

Voici le chemin complet :

```
Vue.js → fetch() → Apache → .htaccess → index.php (router)
       → TaskController.php → database.php (PDO) → MySQL
```

Maintenant, on déroule tout ça **pas à pas**, comme un film.

🎬 1. L'utilisateur clique sur "Ajouter une tâche" dans Vue.js

Dans ton interface :

```
async createTask() {
 const res = await fetch(this.apiBaseUrl, {
   method: 'POST',
   headers: { 'Content-Type': 'application/json' },
   body: JSON.stringify({
     title: this.newTitle,
     done: 0,
   }),
 });
}
```

➡️ Vue.js prépare une requête HTTP :

- **Méthode :** POST
- **URL :** `http://localhost/api/public/tasks`
- **Body JSON :**

```
{ "title": "Nouvelle tâche", "done": 0 }
```

🚦 2. Apache reçoit la requête

Apache reçoit :

```
POST /api/public/tasks
```

Mais Apache ne sait pas encore quoi faire.
C'est le `.htaccess` qui entre en scène.

🔁 3. `.htaccess` réécrit l'URL

Ton `.htaccess` contient :

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]
```

➡️ Apache vérifie :

- `/tasks` n'est **pas** un fichier
- `/tasks` n'est **pas** un dossier

Donc il réécrit la requête :

```
/tasks  →  /index.php
```

Mais il **garde l'URL originale** en mémoire.

🧠 4. `public/index.php` joue le rôle de routeur

Dans `index.php` :

```
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$path = trim($path, '/');
$segments = explode('/', $path);
$resource = $segments[0];
$id = $segments[1] ?? null;
```

Pour `/tasks` :

- `$resource = "tasks"`
- `$id = null`
- `$method = "POST"`

Le routeur décide :

```
if ($resource === 'tasks') {
    switch ($method) {
        case 'POST':
            $controller->store();
            break;
    }
}
```

➡️ Il appelle **TaskController::store()**

🛠️ 5. TaskController reçoit la requête

Dans `store()` :

```
$data = $this->getJsonInput();
```

`getJsonInput()` lit le body JSON :

```
$raw = file_get_contents('php://input');
$data = json_decode($raw, true);
```

➡️ `$data = ["title" => "Nouvelle tâche", "done" => 0]`

Ensuite :

```
$stmt = $pdo->prepare("INSERT INTO tasks (title, done) VALUES (?, ?)");
$stmt->execute([$data['title'], $data['done']]);
```

🗄️ 6. `database.php` fournit la connexion PDO

`getPDO()` crée une connexion MySQL :

```
$pdo = new PDO(
  "mysql:host=localhost;dbname=ma_base;charset=utf8mb4",
  "root",
  "password"
);
```

➡️ Le contrôleur utilise ce PDO pour exécuter le `INSERT`.

🧱 7. MySQL insère la ligne

La requête exécutée :

```
INSERT INTO tasks (title, done)
VALUES ("Nouvelle tâche", 0);
```

MySQL crée une nouvelle ligne :

id	title	done
5	Nouvelle tâche	0

🔙 8. Le contrôleur renvoie la réponse JSON

Après insertion :

```
$id = $pdo->lastInsertId();
$this->show($id);
```

`show()` renvoie :

```
{
  "id": 5,
  "title": "Nouvelle tâche",
  "done": 0
}
```

🖥️ 9. Vue.js reçoit la réponse et met à jour l'interface

Dans ton frontend :

```
const created = await res.json();
this.tasks.push(created);
```

➡️ La nouvelle tâche apparaît instantanément dans la liste.

🎉 Résumé ultra-simple

Étape	Qui agit ?	Ce qui se passe
1	Vue.js	Envoie un POST JSON
2	Apache	Reçoit la requête
3	.htaccess	Redirige vers index.php
4	index.php	Analyse l'URL et appelle le contrôleur
5	TaskController	Lit le JSON, prépare l'INSERT
6	PDO	Exécute la requête SQL
7	MySQL	Insère la ligne
8	TaskController	Retourne la tâche créée
9	Vue.js	Met à jour l'interface

Si tu veux, je peux aussi te faire :

- un **schéma visuel** du flux complet
- une version **orientée objets** du backend

- une version **Vue 3 + Vite + composants**
- un **service API** propre côté frontend

Tu veux aller dans quelle direction ?